BolBahi: Voice Hackathon Build Pack

Tagline: Bol do, bahi ban jayegi. (Speak, and your ledger writes itself.)

This pack is written so you can paste sections straight into an AI code editor (Cursor, Claude Code, Copilot). Section 8 is the master brief to paste first.

1. The Idea

BolBahi is a voice-first digital khata (credit ledger) for kirana and small shop owners in India. A shopkeeper talks in Hindi, English or Hinglish, mid-work, hands busy, and one sentence updates the ledger, stock list and reminders, then the app speaks back the result.

> "Ramesh ne 500 udhaar liya, Sharma ji ne 200 wapas diye, aur atta 10 kilo kam hai, kal Ramesh ko yaad dila dena."

BolBahi extracts **four actions from one sentence**, writes them to MongoDB, sends a real reminder email, updates a live dashboard, and replies: "Ho gaya. Ramesh ka 500 udhaar likha, Sharma ji ka 200 jama, atta low stock mein, Ramesh ko kal ka reminder set."

Why this wins against the rubric

- **Working demo (30%)**: one tight loop (speak, parse, write, speak back) with a live dashboard that visibly changes while the judge watches. No dead screens.
- **Real problem (25%)**: tens of millions of small shops still use paper bahi-khata. Typing is slow, many owners are more comfortable in speech than in forms. Unpaid credit is their biggest cash-flow pain.
- **Smart voice AI (20%)**: multi-intent extraction from a single utterance, Hinglish handling, fuzzy name resolution, spoken clarifying questions, voice queries ("Ramesh ka kitna baaki hai?"), voice undo.
- **Creativity (15%)**: the "one sentence, many actions" moment plus an auto-generated spoken daily summary ("Aaj 1,200 ka udhaar gaya, 800 wapas aaya").
- **Presentation (10%)**: a 3-minute story with a shopkeeper persona and a live before/after.

Be honest (Rule 05)

Say on stage what is real and what is mocked. Real: speech recognition, LLM parsing, DB writes, email reminders, spoken replies. Mocked (if you choose): WhatsApp/SMS delivery (shown as a simulated "message sent" card, email is the real channel), and seeded demo customers.

2. PRD (Product Requirements)

2.1 Problem

Small shopkeepers track credit in paper notebooks. Entries are slow, get lost, disputed, and reminders to debtors are awkward or forgotten. Existing khata apps need typing and menu navigation.

2.2 Target user

Primary persona: Raju Bhaiya, 42, runs a kirana store, speaks Hindi/Hinglish, uses a mid-range Android phone, serves customers while recording entries. Wants: speed, no typing, no mistakes, gentle reminders.

2.3 Goals and non-goals

Goals (hackathon): complete voice loop in under 4 seconds end to end; correct multi-intent extraction on the demo script; visible real actions; works on Chrome (desktop and Android). **Non-goals**: payments, real WhatsApp Business API, offline mode, billing/GST, perfect accuracy on every dialect.

2.4 Core user stories (P0 = must ship by hour 4.5)

ID	Priority	Story	Acceptance
U1	P0	As a shopkeeper I hold a mic button and speak in Hindi/English/Hinglish	Live transcript appears while speaking; works hi-IN and en-IN
U2	P0	I say who took credit and how much	Ledger entry created with customer, amount, type=`udhaar`; balance updates
U3	P0	I say a customer paid	Entry type=`payment`; balance decreases

ID	Priority	Story	Acceptance
U4	P0	I say multiple things in one sentence	All actions executed in order, each shown as an action card
U5	P0	I hear a spoken confirmation	TTS reads a short summary in the same language I used
U6	P0	I set a reminder for a debtor	Reminder doc created; real email sent to customer (or demo inbox) at once or on schedule
U7	P1	I ask a question ("Ramesh ka kitna baaki hai?")	Spoken + visual answer, no DB write
U8	P1	I log low stock ("atta khatam ho raha hai")	Item added to low-stock list
U9	P1	Ambiguous name (two Rameshes)	App asks a spoken clarifying question and listens again
U10	P1	I say "last wala hata do"	Last action group is soft-deleted (undo)
U11	P1	I ask "aaj ka hisaab"	Spoken daily summary + summary card
U12	P2	Dashboard updates live	New entries appear without refresh (SSE)
U13	P2	Customer statement page	Shareable read-only link `/s/:token`

2.5 Feature list

1. **Hold-to-talk voice console** with waveform and live transcript.
2. **Multi-intent engine** (LLM tool-calling) returning an ordered list of actions.
3. **Ledger**: customers, entries, running balances.
4. **Reminders**: email via provider, simulated WhatsApp card, scheduled via node-cron.
5. **Stock watch**: low-stock list.
6. **Voice queries and summaries.**
7. **Clarification loop** for ambiguity and low confidence.
8. **Undo** by voice.

9. **Live dashboard**: total outstanding, today in/out, top debtors, activity feed.
10. **Language toggle** (Hinglish default, English) that also sets TTS voice.
11. **Demo mode**: seeded data plus a "Run demo script" button (type the sentence if the mic fails; judges forgive this, they don't forgive a dead demo).

2.6 Success metrics (for your own testing)

- Demo sentence parsed 9/10 times correctly.
- Time from mic release to first spoken word under 4 s.
- Zero unhandled errors in a 3-minute run.

2.7 Detailed functional specification

This section is the source of truth for behavior. If the code and this section disagree, fix the code.

2.7.1 Intent catalog

Intent	Example utterances	Fields extracted	System behavior	Reply template
`add_udhaar`	"Ramesh ne 500 udhaar liya", "Sharma ji ko do sau ka maal diya, likh lo"	customer, amount, note?	Create Entry (udhaar), recompute balance	"Ramesh ka 500 udhaar likh diya. Total baaki 1,200."
`add_payment`	"Sharma ji ne 200 wapas diye", "Gupta ji ne paise de diye, 1000"	customer, amount	Create Entry (payment). If it exceeds the balance, record it and mention the advance	"Sharma ji ka 200 jama. Ab baaki 300."
`set_reminder`	"kal Ramesh ko yaad dila dena", "Friday ko Suresh ko reminder bhej do"	customer, when, message?	Create Reminder; send email now or via cron at `when`	"Ramesh ko kal subah 9 baje reminder bhejunga."

Intent	Example utterances	Fields extracted	System behavior	Reply template
`add_low_stock`	"atta 10 kilo kam hai", "chini khatam ho rahi hai"	item, quantityHint?	Upsert StockItem as low	"Atta low stock mein daal diya."
`query_balance`	"Ramesh ka kitna baaki hai?", "Sharma ji ka hisaab"	customer	Read only, no writes	"Ramesh ka 1,200 baaki hai."
`daily_summary`	"aaj ka hisaab", "aaj kitna udhaar gaya"	none	Aggregate today's entries	"Aaj 1,200 udhaar gaya, 800 wapas aaya."
`undo_last`	"last wala hata do", "galat ho gaya"	none	Soft delete the newest groupId, recompute balances	"Pichla entry hata diya."
`clarify`	any ambiguous or incomplete utterance	question, options?	Store pending state, speak the question, auto-listen	"Kaunsa Suresh? Gupta ya Yadav?"

Keep every reply template in two forms: Roman Hinglish for the screen and Devanagari for `hi-IN` text to speech, since Devanagari reads far more naturally in Hindi voices. English replies use the `en-IN` voice.

2.7.2 Behavior rules per feature

- **Voice capture**: hold to talk; release or 1.5 s of silence ends capture; interim text is grey and final text is dark; a transcript under 3 characters is ignored silently. A typed input box submits through the exact same endpoint.
- **Parsing**: one command may hold up to 8 actions, executed in spoken order. Numbers in words ("paanch sau", "dedh hazaar", "do hazaar paanch sau") become integers in INR. No amount means a `clarify`, never a guess.

- **Customer resolution**: exact name, then alias, then fuzzy match. Honorifics ("ji", "bhaiya", "sahab") are stripped before matching. An unknown name on `add_udhaar` creates a new customer and the reply says so ("Naya customer jodh diya"). An unknown name on a payment, reminder or query asks "Kaun sa customer? Naam dobara bolein."
- **Ledger**: entries are append-only; corrections happen through undo, never by editing. Balance is always recomputed from non-deleted entries. Show negative balances as "advance".
- **Reminders**: default time for "kal" is 9:00 AM IST; "shaam ko" is 6:00 PM; "abhi" sends now. The email states the customer's name, the pending amount and the shop name. A customer with no email uses `DEMO_EMAIL`, and the action card says so.
- **Stock**: same item name (case and spelling insensitive) updates the existing row instead of duplicating it. Marking "ok" again is allowed by voice ("atta aa gaya").
- **Queries and summaries**: strictly read only. They never create a groupId, so undo cannot remove them.
- **Undo**: removes only the latest write command. Say "phir se hata do" for the one before. Reminders already emailed are marked `cancelled` but cannot be recalled, and the reply states that.
- **Language toggle**: sets recognition language, TTS voice and reply language together. If the user speaks English while Hinglish is selected, the reply follows the detected language.
- **Dashboard**: updates through SSE within 1 second of a write, highlights the changed row for 2 seconds, and counts numbers up.

2.7.3 Conversation flow

1. **Idle**: mic button shows "Dabake bolo".
2. **Listening**: pulse animation and live transcript. Release the mic to move on; a 10 s cap ends capture automatically.
3. **Thinking**: show "Soch raha hoon..." with skeleton action cards. Hard timeout 8 s.
4. **Branch A, clear command**: Executing, then Speaking the reply, then Idle.
5. **Branch B, clarification needed**: Speaking the question, then auto Listening (no button press). If no answer arrives within 8 s or the user says "rehne do", cancel and return to Idle. A valid answer re-enters Thinking with the pending question attached.
6. **Branch C, error**: Speaking the friendly error, showing a Retry button, then Idle. Nothing is written on a failed parse.

7. **Interruption**: pressing the mic while speaking stops TTS immediately and starts Listening.

2.7.4 Edge cases

Situation	Expected behavior
Two customers named Suresh	Ask which one, offering last names or areas as options
"Ramesh ne 500 liye aur 200 de diye" in one sentence	Two entries on the same customer, both shown, one combined spoken reply
Amount missing ("Ramesh ne udhaar liya")	Ask "Kitna udhaar?" and listen again
Speech misheard a name ("Rakesh" for "Ramesh")	Fuzzy match proposes the nearest name and asks to confirm only if the score is below the confidence bar
Transcript is background noise	Ignore silently, show "Sunai nahi diya, dobara bolein"
Gemini rate limit or outage	Resilience chain in 3.1b; the UI never shows a raw error
Mic permission denied	Show a one-tap instruction card and keep typed input working
Browser has no Web Speech support	Show "Chrome use karein" banner and typed input
Same command sent twice (double tap)	Idempotency key returns the first result, nothing written twice
Amount above 1,000,000 or negative	Reject and ask the user to confirm the amount

2.8 Accounts and multi-shop data isolation

Every shopkeeper registers their own account and shop, and no query ever crosses from one shop to another. This is a P0 feature: it makes the app a real product instead of a single-user toy, and it gives you a strong demo moment.

ID	Priority	Story	Acceptance
A1	P0	Register with shop name, owner name, email or phone, and password	Shop and User created, auth cookie set, user lands on an empty Home with a "Demo data bharein" offer
A2	P0	Login with identifier and password	Valid credentials set the cookie and open Home; invalid ones show one generic message ("Email ya password galat"), never which part was wrong
A3	P0	My data stays mine	Another shop's customer, entry, stock, reminder or command id returns 404 (not 403); the SSE stream carries only my shop's events; the LLM prompt contains only my customers
A4	P0	Stay signed in, and sign out	Session lasts 7 days; logout clears the cookie; an expired session redirects to login and returns to the page I wanted
A5	P1	Seed demo data for my shop	Settings button resets and seeds only the current shop
A6	P1	My language preference is remembered	Stored on the Shop, applied at login
A7	P1	Brute-force protection	After 5 failed logins in 15 minutes from one IP, show "Thodi der baad try karein"

Form rules

- Register fields: shop name, owner name, email or 10-digit Indian mobile number, password (8 characters or more), with show/hide password. Login fields: identifier and password.
- Inline validation in the user's language; big 56 px inputs, one column, glass card centered on a soft gradient (design in 4.8).
- After register, log in automatically; never ask the user to log in twice.

- A language toggle is visible on both forms, because many users will read Hindi first.

Non-goals for this build: password reset, social login, staff accounts. A shop has one owner login, but `User` is a separate model from `Shop` so staff accounts can be added later.

Demo proof (15 seconds): pre-create two accounts, "Raju General Store" with the full demo data and "Meena Kirana" with a few different customers. During the demo, log out, log in as Meena, show different customers and totals, and say "data never mixes between shops." Then log back into Raju for the main demo.

Time cost: budget about 45 minutes for auth and route guards. Build it in step 1 of the master brief so every later feature is shop-scoped from the start; retrofitting `shopId` later is the most common way to get a leak.

2.9 Home launcher and screens

There are no tabs. After login, Home is an orbital carousel: one huge glassmorphic tile is visible at a time, and the user swipes (phones), taps the arrow buttons, or uses the keyboard arrows to swing the next tile into view along an elliptical orbit. Tapping the active tile opens that screen as a full-screen glass panel with a back chip. The mic tile is the default and works directly from Home, so the core loop never needs navigation. Motion, sizes and formulas are in 4.8.2.

Priority	Tile (Hinglish label)	Opens	Live badge on the tile
P0	**Mic** ("Bolo")	Hold to talk right on Home; a result sheet slides up with transcript, action cards and the spoken reply	Pulse while listening
P0	**Money Records** ("Paise ka Khata")	Chronological feed of every udhaar and payment across all customers	Today's net (for example +₹600)
P0	**Customers** ("Grahak Khata")	Searchable customer cards with balances; tap for that customer's ledger	Total outstanding

Priority	Tile (Hinglish label)	Opens	Live badge on the tile
P0	**Commands** ("Meri Awaazein")	History of everything said, with an option to delete any command	Count of today's commands
P1	**Today's Summary** ("Aaj ka Hisaab")	Big-number summary of today plus a "Sunao" button that reads it aloud	Net of today
P1	**Stock** ("Maal")	Low-stock list, tap to mark "aa gaya"	Low-stock count
P1	**Reminders** ("Yaad-dihani")	Pending and sent reminders, cancel a pending one, resend	Pending count
P1	**Settings** (small glass icon in the top bar, not a tile)	Shop name, language, big text, demo data, logout	none

The tiles beyond your four (summary, stock, reminders, settings) are my additions: they turn the voice features already in the spec into places the shopkeeper can actually see and manage, and they cost little because the data and APIs already exist.

2.9.1 Home screen

- Top bar: shop name on the left, language toggle (हि / EN) and a settings icon on the right. Under it, one line of greeting ("Namaste, Raju ji").
- The mic tile is first and is the default; swipe or arrows move through the other tiles in the order above and loop back to the mic.
- Each tile: huge icon (about 88 px), label in the user's language, live badge, subtle "glow" in the tile's semantic color.
- A rotating example chip under the mic ("Bolo: Ramesh ne 500 udhaar liya") teaches new users what to say.
- Offline or API down: tiles still render; badges show a dash and a small "Connect nahi hua" banner appears.

2.9.2 Money Records screen

- Feed grouped by day ("Aaj", "Kal", dates). Each row: customer initial avatar, name, note, time, and an amount with sign and color (+ green payment, - red udhaar). A small chip shows the command it came from.
- Filters as glass chips: Sab, Udhaar, Jama, and ranges Aaj, Hafta, Mahina. A search box filters by customer name.
- Tap a row: bottom sheet with the details and a link to "Original command", which jumps to that item in Commands.
- Header summary: total in, total out, net for the selected range.
- P2: Export CSV for the selected range.
- Empty state: "Abhi koi hisaab nahi. Mic dabakar bolein" with a tappable example.

2.9.3 Customers screen

- Cards sorted by highest amount owed. Each card: avatar, name, last activity ("3 din pehle"), balance in red (owed) or green (advance), and a mini tag if a reminder is pending.
- Search by name (typed, or by tapping the mic inside the search box).
- Tap a card: right-side drawer (bottom sheet on mobile) with the customer's ledger, running balance, and buttons "Yaad dilao" (sends a reminder now) and, in P2, "Statement share".
- Add or edit email and phone in the drawer, so reminders have a real destination.
- P2: "Sabko yaad dilao" bulk action for customers who owe money and have been quiet for 7 or more days.

2.9.4 Commands screen (history with delete)

- Newest first, grouped by day. Each row: the raw transcript in quotes, chips for each understood action ("Ramesh +₹500", "Reminder kal"), time, and a status (Done, Partly done, Failed, Deleted).
- Tap to expand: full parsed actions, results per action, which model handled it (Gemini or fallback), and latency. This doubles as proof of the AI working during the demo.
- **Delete**: trash icon on each row opens a glass confirm sheet: "Is command ko hata dein? Isse jude 2 entries bhi hat jayenge." On confirm, the command and every effect it produced are undone (details in 3.15). The row animates out; balances update live.
- Failed commands show a Retry button that resubmits the same transcript.

- P2: select several rows and delete together; "Edit and resend" lets the user fix a misheard word.
- Empty state: "Abhi tak kuch nahi bola. Mic dabakar shuru karein."

2.9.5 Today's Summary, Stock, Reminders, Settings

- **Today's Summary**: three big numbers (udhaar gaya, wapas aaya, net), the top 3 people who owe the most, and a "Sunao" button that speaks the same summary as the voice query.
- **Stock**: list of items marked low with the time they were noted; tap "Aa gaya" to mark ok; a quick share/copy of the list for the supplier (P2).
- **Reminders**: tabs-free list sections (Aane wale, Bheje gaye); each row shows customer, channel (Email, with a "Demo" chip on simulated WhatsApp), time and status; actions Cancel (pending) and Resend (sent or failed).
- **Settings sheet**: shop name, owner name, language, big text toggle (125 percent), "Demo data bharein" (resets this shop only), logout. Nothing else.

2.9.6 Build priority and cut order

Build P0 tiles first: Mic, Money Records, Customers, Commands. If time runs short, cut in this order: bulk remind and CSV export, Stock screen, Reminders screen (keep the data, hide the tile), Today's Summary screen (the voice query still works). Never cut Commands delete: it is the visible control that tells judges the user owns their data.

3. TRD (Technical Requirements)

3.1 Stack

Layer	Choice	Reason
Frontend	**Next.js 14+ (App Router), TypeScript, Tailwind, shadcn/ui, Framer Motion, lucide-react**	Fast UI, polished defaults
Backend	**Express.js on Node 20, TypeScript, Mongoose, zod**	Fixed by you; zod validates LLM JSON
Database	**MongoDB Atlas (free M0)**	Fixed by you

Layer	Choice	Reason
STT	**Browser Web Speech API** (`webkitSpeechRecognition`, lang `hi-IN` / `en-IN`)	Zero setup, supports Hinglish reasonably. Fallback (stretch): Deepgram via MediaRecorder
LLM	**`Google Gemini Flash (free tier) via @google/genai, using forced function calling. Model name read from GEMINI_MODEL env`**	Free for prototyping, fast, supports structured output. Behind an LLMProvider interface so Claude can be swapped in with one env flag
TTS	**Browser SpeechSynthesis** (hi-IN voice). Stretch: ElevenLabs	Free, instant
Email	**Resend** (free tier) or Nodemailer + Gmail app password	Real, visible action
Scheduling	**node-cron**	Simple in-process scheduler
Realtime	**Server-Sent Events** (`/api/events`)	Simpler than WebSockets
Deploy	Vercel (web), Render (API). Keep local as backup	Judges may want a URL
Auth	Email or phone + password, bcryptjs hashing, JWT in an httpOnly cookie, zod validation	Keeps each shopkeeper's data separate; simple and fast to build

Note: the Web Speech API needs HTTPS (or localhost) and works best in Chrome/Edge. Demo on Chrome.

3.1b LLM provider strategy (Gemini first)

Gemini Flash is the primary model because Google's free tier covers the Flash and Flash-Lite models without a card. Treat any rate-limit number you read online as approximate: limits differ by model and change often, so check Google AI Studio for your key's real quota before the event.

Two constraints shape the design:

- **Free-tier data is used by Google** to improve its products, so send only the transcript and first names. Never send phone numbers, emails or real customer data, and demo with fake data.

- **Quota is small**, so every call must count: one call per voice command, a small prompt, and a short cache for repeated transcripts.

Put all model access behind one interface so the provider is a config flag, not a rewrite:

```
// apps/api/src/services/llm/provider.ts
export interface LLMProvider {
  name: 'gemini' | 'claude';
  parse(input: {
    transcript: string;
    lang: 'hi-IN' | 'en-IN';
    nowIso: string;          // Asia/Kolkata
    customers: string[];     // top 30 first names only
    history: string[];       // last 3 transcripts
    pendingQuestion?: string // set during a clarification turn
  }): Promise<ParseResult>;
}

// apps/api/src/services/llm/gemini.ts
import { GoogleGenAI } from '@google/genai';
const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY! });

const res = await ai.models.generateContent({
  model: process.env.GEMINI_MODEL!,
  contents: userPrompt,
  config: {
    systemInstruction: SYSTEM_PROMPT,
    temperature: 0.1,
    tools: [{ functionDeclarations: [recordActionsDecl] }],
    toolConfig: { functionCallingConfig: { mode: 'ANY', allowedFunctionNames:
['record_actions'] } },
  },
});
const args = res.functionCalls?.[0]?.args; // then zod.parse(args)
```

If the TypeScript types complain about `mode`, use the `FunctionCallingConfigMode.ANY` enum exported by the SDK.

Resilience chain (stop at the first success, and always tell the user honestly what ran):

1. Primary model (`GEMINI_MODEL`).
2. Retry once after a 400 ms backoff on 429 or 503.

3. Secondary model (`GEMINI_FALLBACK_MODEL` , a Flash-Lite model). Add this variable to `.env` .
4. Rule-based fallback parser for the simplest patterns only (name + amount + udhaar or payment word). Label results from it as `source: 'fallback'` in the activity feed and mention it in the demo if it fires.
5. Spoken error: "Samajh nahi aaya, dobara bolein."

Quota savers: cache the parsed result of an identical transcript for 60 seconds (helps rehearsals), send only the top 30 names, keep the system prompt under about 600 tokens, and never make a second LLM call just to phrase the reply; use templates.

Optional Claude swap: implement `claude.ts` against the same interface with tool use and set `LLM_PROVIDER=claude` . Run the 7-step test script against both and keep whichever handles Hinglish numbers better. Claude's API has no ongoing free tier, only a small trial credit for new accounts, so Gemini stays the default.

3.2 Architecture

```
[Mic] -> Web Speech API -> transcript (client)
   -> POST /api/voice/command {transcript, lang, sessionId}
        -> Express: build context (customer names, recent entries)
        -> Gemini (function calling) -> actions[] (zod-validated)
        -> Executor: run each action (Mongo writes, email, etc.)
        -> Compose spoken reply (second small LLM call OR template)
   <- {actions[], results[], reply, needsClarification?, followUp?}
-> UI renders action cards + SpeechSynthesis speaks reply
-> SSE pushes dashboard updates
```

Keep it **one LLM call** for parsing plus **templated replies** (fast, reliable). Optionally one more LLM call only to phrase the reply in natural Hinglish.

3.3 Folder structure (monorepo)

```
bolbahi/
  apps/
    web/                 # Next.js
      app/ (login/, register/, (app)/home, (app)/records, (app)/customers,
(app)/commands, (app)/stock, (app)/reminders, (app)/hisaab, (app)/settings,
s/[token]/)
      components/ (GlassTile, TileGrid, GlassPanel, MicButton, Transcript,
ActionCard, RecordRow, CustomerCard, CommandRow, AuthForm)
      lib/ (speech.ts, tts.ts, api.ts)
    api/                 # Express
      src/
        index.ts
        routes/ (auth.ts, voice.ts, commands.ts, customers.ts, entries.ts,
stock.ts, reminders.ts, stats.ts, events.ts, demo.ts)
        services/ (llm.ts, executor.ts, resolver.ts, reminders.ts, mailer.ts,
summary.ts, replies.ts)
        models/ (User.ts, Shop.ts, Customer.ts, Entry.ts, Reminder.ts,
StockItem.ts, ActionLog.ts)
        schemas/ (actions.ts)   # zod
        lib/ (db.ts, sse.ts, env.ts, jwt.ts), middleware/ (requireAuth.ts,
errorHandler.ts)
  .env.example
  README.md
```

3.4 Data models (Mongoose)

Shop: `_id, name, ownerName, language: 'hi-IN' | 'en-IN', createdAt`

User: `_id, shopId, identifier (email or phone, unique), passwordHash, createdAt`

Rule: every model below carries a required, indexed `shopId`, and every query filters by it. See 3.14.

Customer: `_id, shopId, name, aliases[string], phone?, email?, balance:number (derived, kept in sync), archivedAt?, createdAt`

Entry: `_id, shopId, customerId, type: 'udhaar'|'payment', amount:number, note?, groupId (one per voice command), deletedAt?, createdAt`

Reminder: `_id, shopId, customerId, message, channel:'email'|'whatsapp_sim', sendAt:Date, status:'pending'|'sent'|'failed'|'cancelled', groupId`

StockItem: `_id, shopId, name, quantityHint?, status:'low'|'ok', groupId, createdAt`

ActionLog (the command history): `_id, shopId, groupId, clientCommandId, transcript, lang, parsedActions(JSON), results(JSON), source:'gemini'|'fallback'|'typed', latencyMs, deletedAt?, createdAt` (powers the Commands History screen, undo and debugging; `{shopId, clientCommandId}` is unique).

Balance rule: `balance = sum(udhaar) - sum(payment)` over non-deleted entries. Recompute after each write rather than incrementing, to keep undo safe.

3.5 Action schema (zod, the contract between LLM and executor)

```
type Action =
 | { type: 'add_udhaar'; customer: string; amount: number; note?: string }
 | { type: 'add_payment'; customer: string; amount: number; note?: string }
 | { type: 'set_reminder'; customer: string; when: string /* ISO or 'now' */;
message?: string }
 | { type: 'add_low_stock'; item: string; quantityHint?: string }
 | { type: 'query_balance'; customer: string }
 | { type: 'daily_summary' }
 | { type: 'undo_last' }
 | { type: 'clarify'; question: string; options?: string[] };

type ParseResult = { language: 'hi'|'en'|'hinglish'; actions: Action[];
confidence: number };
```

Implement with **Gemini function calling**: declare one function `record_actions` whose parameters follow the schema above, force it with `toolConfig.functionCallingConfig` set to mode `ANY` and `allowedFunctionNames: ['record_actions']`, then validate the returned args with zod. Gemini's schema subset handles unions poorly, so declare `actions` as an array of ONE flat object (a `type` enum plus optional fields like `customer`, `amount`, `when`, `item`) and let zod discriminate on `type`. Set temperature to 0.1. If function calling misbehaves, fall back to JSON mode (`responseMimeType: 'application/json'` with a `responseSchema`).

3.6 LLM system prompt (starting point)

```
You are the parser for BolBahi, a voice ledger for Indian kirana shops.
Input is a speech transcript in Hindi, English or Hinglish (may contain
recognition errors).
Return ALL actions in the sentence, in spoken order, by calling
record_actions.
Rules:
- 'udhaar liya/diya, likh do, baaki' => add_udhaar. 'wapas diye, jama, paid,
de diye' => add_payment.
- Convert number words and Hindi numerals ('paanch sau', 'dedh hazaar', 'do
hazaar') to integers (INR).
- Resolve relative time ('kal', 'parson', 'Friday', 'shaam ko') against NOW=
{{now_iso}} in Asia/Kolkata.
- Use the closest name from KNOWN_CUSTOMERS={{names}}; if two match equally,
emit clarify.
- Never invent amounts. If an amount is missing, emit clarify.
- 'hata do/undo/galat ho gaya' => undo_last.
- Questions ('kitna baaki', 'hisaab') => query_balance or daily_summary,
never a write.
```

Send the last 3 transcripts as context so "usko 100 aur likh do" can resolve.

3.7 Executor rules

- Run actions **sequentially**, collect a result object per action, never abort the whole batch on one failure.
- Generate one `groupId` per command; undo soft-deletes everything with the latest `groupId`.
- **Resolver**: exact match, then alias match, then fuzzy (Fuse.js, threshold 0.35). Unknown name on `add_udhaar` creates the customer. Multiple strong matches returns `needsClarification`.
- **Clarification state**: store `pendingClarification` in memory keyed by `sessionId` with a 60 s TTL. The next transcript is sent to the LLM along with the pending question.
- **Reminders**: if `when` is `now` or within 1 minute, send immediately; else insert pending and let node-cron (every 30 s) dispatch. Email to `customer.email`, or to `DEMO_EMAIL` env var if missing.
- Emit an SSE event `ledger.updated` after every command.

3.8 API contract

Method	Route	Purpose
POST	`/api/auth/register`	Create Shop + User, set auth cookie. Body `{shopName, ownerName, identifier, password}`
POST	`/api/auth/login`	Verify credentials, set auth cookie
POST	`/api/auth/logout`	Clear the cookie
GET	`/api/auth/me`	Current user and shop; called on every app load
POST	`/api/voice/command`	Main loop. Body `{transcript, lang, sessionId, clientCommandId}`
GET	`/api/commands`	Command history, newest first, paged
DELETE	`/api/commands/:groupId`	Delete a command and undo its effects (see 3.15)
GET	`/api/stats`	Outstanding total, today in/out, top 5 debtors, low-stock count, pending reminders
GET	`/api/entries`	Records feed across all customers; filters `type`, `from`, `to`, `customerId`
GET	`/api/customers`	List with balances; `?q=` search
GET	`/api/customers/:id/entries`	Ledger for one customer
GET / PATCH	`/api/stock`, `/api/stock/:id`	List low-stock items; mark an item ok
GET / DELETE	`/api/reminders`, `/api/reminders/:id`	List reminders; cancel a pending one
GET	`/api/events`	SSE stream for the logged-in shop only
POST	`/api/demo/seed`	Reset and seed demo data for the current shop

Method	Route	Purpose
GET	`/api/statement/:token`	Public read-only customer statement (P2)

Every route except register, login and the public statement requires a valid auth cookie.

Response of `/voice/command`:

```
{ "groupId":"...", "actions":[...], "results":[{"ok":true,"summary":"Ramesh: +500 udhaar"}],
  "reply":"Ho gaya. Ramesh ka 500 udhaar likha...", "lang":"hi-IN",
  "needsClarification":false, "latencyMs":1900 }
```

3.9 Frontend technical notes

- `speech.ts`: wrap `SpeechRecognition` with `interimResults=true`, `continuous=false`; hold-to-talk using pointer events; auto-stop after 1.5 s silence. Expose `start(lang)`, `stop()`, callbacks `onInterim`, `onFinal`, `onError`.
- `tts.ts`: pick a `hi-IN` voice if available, else `en-IN`; cancel any ongoing speech before speaking; **pause recognition while speaking** to avoid the app hearing itself.
- Show a typed-input fallback and a "Run demo sentence" button.
- Use SSE via `EventSource` and refetch `/api/stats` on events.
- CORS: allow the web origin only. Env: `NEXT_PUBLIC_API_URL`.

3.10 Environment variables

`MONGODB_URI, GEMINI_API_KEY, GEMINI_MODEL, LLM_PROVIDER=gemini, ANTHROPIC_API_KEY (optional, only if LLM_PROVIDER=claude), ANTHROPIC_MODEL=claude-haiku-5-5, RESEND_API_KEY, MAIL_FROM, DEMO_EMAIL, JWT_SECRET, JWT_EXPIRES_IN=7d, COOKIE_SECURE, PORT=4000, WEB_ORIGIN, NEXT_PUBLIC_API_URL`

Set `GEMINI_MODEL` to the current free-tier Flash model listed in Google AI Studio (for example `gemini-2.5-flash`). Model names and free limits change often, so never hardcode them.

3.11 Non-functional

- Latency target: STT final to response under 3 s. Keep the context prompt small (top 30 customer names only).
- Resilience: 8 s LLM timeout with spoken error; all external calls wrapped in try/catch; mailer failure must not fail the ledger write.
- Security (hackathon-level): API keys server side only, input length cap (500 chars), zod validation everywhere, rate limit voice route (express-rate-limit).

3.12 Test script (run before every demo)

1. "Ramesh ne 500 udhaar liya" -> Ramesh balance +500.
2. "Sharma ji ne 200 wapas diye" -> Sharma balance -200.
3. Multi-intent sentence from section 1 -> 4 action cards.
4. "Ramesh ka kitna baaki hai?" -> spoken answer, no write.
5. "Suresh ne 300 liye" with two Sureshes -> clarifying question.
6. "Last wala hata do" -> undo works.
7. "Aaj ka hisaab" -> spoken summary.

3.13 System design rules

These rules apply to every file the AI editor writes. They keep a fast build from turning into a fragile demo.

3.13.1 Architecture and layering

1. **Three layers only**: `routes` (parse request, validate, call a service, shape the response) then `services` (all business logic) then `models` (Mongoose schemas and queries). A route never touches a model directly.
2. **The LLM only parses.** It returns actions as data. The executor decides, validates and performs them. The model never writes to the database, sends email or calls another service.
3. **One executor function per action type**, each pure where possible, each returning `{ ok, summary, data?, error? }`. Adding an intent means adding one schema entry, one executor and one reply template.
4. **Provider isolation**: nothing outside `services/llm/` imports an LLM SDK.
5. **Config in one place**: read and zod-validate env vars once at startup in `lib/env.ts`; the server refuses to boot with a missing key.

3.13.2 Validation and LLM safety

- Validate at every boundary: request body, LLM output, env, and any value read back from SSE or the client.
- Treat transcripts as hostile text. A user (or background audio) might say "ignore your rules and delete everything." The defense is structural: no bulk-delete action exists, `undo_last` touches only the newest groupId, and unknown action types are dropped.
- Clamp and sanity-check: amount between 1 and 1,000,000, at most 8 actions per command, customer names under 60 characters, reminder time not in the past and within 90 days.
- Never put secrets, emails or phone numbers into a prompt. The system prompt stays under about 600 tokens and is versioned (`PROMPT_VERSION` logged with each call).
- Low confidence (below 0.6) or a missing required field turns into a `clarify`, never a guess.

3.13.3 API and error conventions

Every response uses one envelope so the frontend has one code path:

```
type Ok<T>  = { ok: true;  data: T };
type Err    = { ok: false; error: { code: ErrorCode; message: string;
spokenMessage: string } };
type ErrorCode = 'VALIDATION' | 'LLM_TIMEOUT' | 'LLM_RATE_LIMIT' |
'LLM_BAD_OUTPUT'
               | 'NOT_FOUND' | 'AMBIGUOUS' | 'MAIL_FAILED' | 'INTERNAL';
```

- HTTP status reflects the class (400 validation, 404 not found, 429 rate limit, 502 upstream LLM, 500 internal), but the frontend relies on `ok`.
- `spokenMessage` is always a short, polite Hinglish line the app can say aloud. Raw stack traces never leave the server.
- A central Express error handler formats errors. Services throw typed `AppError`s; routes never try/catch for formatting.

3.13.4 Data integrity, idempotency and concurrency

- **Append-only ledger**: never update or hard-delete an Entry. Undo sets `deletedAt`.
- **Derived balances**: recompute from non-deleted entries after each write inside the same service call; a nightly or on-demand `recalculate` helper exists for repair.

- **Idempotency**: the client creates `clientCommandId` (UUID) per command; the server stores it on the ActionLog with a unique index and returns the stored result on a repeat.
- **Per-session serialization**: process one command at a time per `sessionId` with a small in-memory promise chain, so two fast utterances cannot interleave writes or clarification state.
- **Group atomicity**: all writes of one command share a `groupId`. If a later action fails, earlier ones stay and the failure is reported per action, because partial success is more useful to a shopkeeper than rolling back a spoken sentence.
- **Indexes**: Customer `name` (text), Entry `{ customerId, deletedAt, createdAt }`, ActionLog `clientCommandId` (unique), Reminder `{ status, sendAt }`.

3.13.5 Performance budget

Stage	Budget	Rule
Speech final to request sent	50 ms	No extra processing on the client
Context build (names, history)	80 ms	Top 30 customers, indexed, cached 10 s
LLM call	1,500 ms target, 8,000 ms hard timeout	One call per command, temperature 0.1
Executor writes	300 ms	Sequential, indexed queries only
Response to first spoken word	4,000 ms total	Start TTS as soon as `reply` arrives, before the dashboard finishes animating
SSE to dashboard update	1,000 ms	Send a small event, let the client refetch `/api/stats`

3.13.6 Realtime (SSE) rules

- One `/api/events` stream per browser tab, heartbeat comment every 20 s, client auto-reconnects with backoff.
- Events carry only a type and an id (`ledger.updated`, `reminder.sent`), never full payloads.
- Close the stream on server shutdown; keep a list of open responses for cleanup.

3.13.7 Security and privacy

- Secrets only in server env; `.env` is git-ignored and `.env.example` lists names only.
- CORS limited to `WEB_ORIGIN`; `express-rate-limit` on `/api/voice/command` (for example 30 requests per minute per IP); body size limit 10 KB.
- Auth is an httpOnly JWT cookie and every query is scoped by shopId (see 3.14). It is still a hackathon build, so avoid real customer data and say so in the README.
- Public statement links (P2) use unguessable tokens (at least 24 random characters), expire after 7 days and show read-only data.

3.13.8 Observability

- One JSON log line per command: `groupId`, `provider`, `model`, `promptVersion`, `latencyMs`, `actionTypes`, `ok`, `errorCode`.
- A hidden `/debug` page (dev only) lists the last 20 ActionLogs with raw transcript, parsed actions and results. This is your fastest way to fix misheard Hinglish during testing.
- Log LLM failures with the retry count; never log API keys or full email addresses.

3.13.9 Code quality and testing

- TypeScript strict mode, no `any` outside LLM boundary adapters, ESLint plus Prettier.
- Small files (under 200 lines), named exports, one responsibility per service.
- Comments only for non-obvious decisions (why, not what).
- Test pyramid for 6 hours: (1) unit tests for the resolver, number-word conversion and reply templates; (2) one script that replays 15 saved Hinglish transcripts against the parser; (3) the manual 7-step test script in 3.12 before every demo.
- Commit after each build step in the master brief so you can roll back a bad AI-generated change quickly.

3.13.10 Deployment rules

- Keep a local run as the primary demo path; deploy web to Vercel and API to Render only after the core loop works.
- Render's free instances sleep, so hit `/health` five minutes before presenting.
- Pin Node 20, commit the lockfile, and include one `npm run dev` at the repo root that starts both apps.

3.14 Auth and multi-tenancy rules

Goal: two shopkeepers can use the same deployment and never see, change or influence each other's data.

3.14.1 Auth flow

1. **Register**: validate with zod, hash the password with `bcryptjs` (cost 10), create the Shop, then the User with that `shopId`, sign a JWT `{ sub: userId, shopId }`, and set it as a cookie.
2. **Login**: look up by `identifier`; always run a bcrypt compare (against a dummy hash if the user doesn't exist) so response time doesn't reveal which identifiers exist; return one generic error on any failure.
3. **Cookie**: `httpOnly`, `sameSite: 'lax'`, `secure` in production, `maxAge` 7 days, name `bb_session`. The JWT secret is at least 32 random characters from `JWT_SECRET`.
4. **requireAuth middleware**: verifies the cookie, sets `req.auth = { userId, shopId }`, otherwise returns `401 AUTH_REQUIRED`. Add `AUTH_REQUIRED` and `AUTH_INVALID` to the error codes in 3.13.3.
5. **Logout**: clears the cookie. `/api/auth/me` returns `{ user, shop }` and is the single source of truth on app load.
6. **Rate limit** login and register separately: 5 failures per 15 minutes per IP on login.

Cookie and domain rule: if web (Vercel) and API (Render) sit on different domains, browsers treat the cookie as third-party and may block it. Avoid the problem by proxying: add a Next.js `rewrites()` entry that forwards `/api/:path*` to the Express URL, so the browser only ever talks to one origin. Locally it is the same origin story, so nothing changes between dev and prod. If you must go cross-origin, use `sameSite: 'none'`, `secure: true`, CORS `credentials: true` and `fetch(..., { credentials: 'include' })`.

3.14.2 The tenant rule

Every service function takes `shopId` as its first argument, and every database call includes it.

```
// WRONG: any logged-in user could read any customer by guessing an id
Customer.findById(id);

// RIGHT: the id only resolves inside my own shop
Customer.findOne({ _id: id, shopId });
```

- `shopId` comes only from `req.auth`, never from the body, query string or URL. Strip any `shopId` field a client sends.
- A resource that exists but belongs to another shop returns **404**, identical to "not found", so ids can't be probed.
- Add a Mongoose safety-net plugin on every tenant model that throws in non-production when a `find`, `findOne`, `updateMany` or `deleteMany` filter lacks `shopId`. It turns a silent leak into a loud test failure:

```
export function tenantGuard(schema: Schema) {
  const ops = ['find', 'findOne', 'findOneAndUpdate', 'updateOne',
'updateMany', 'deleteOne', 'deleteMany', 'countDocuments'];
  ops.forEach((op) => schema.pre(op as any, function (this: any) {
    if (process.env.NODE_ENV !== 'production' && !('shopId' in
this.getFilter())) {
      throw new Error(`Missing shopId in ${op} on ${this.model.modelName}`);
    }
  }));
}
```

3.14.3 Where tenancy must be enforced

Area	Rule
Indexes (supersedes 3.13.4)	Every compound index starts with `shopId`: Customer `{shopId, name}`, Entry `{shopId, customerId, deletedAt, createdAt}`, Reminder `{shopId, status, sendAt}`, ActionLog `{shopId, clientCommandId}` unique, User `identifier` unique
LLM context	Customer names in the prompt come only from the current shop
Resolver	Fuzzy search runs only over the current shop's customers
Clarification state and command queue	Keyed by `shopId:sessionId`
SSE	Connections are stored per `shopId`; an event is sent only to that shop's connections

Area	Rule
Reminder cron	One global job selects pending reminders across shops, but loads each shop's data by its own `shopId`, and emails show that shop's name
Seed and reset	`/api/demo/seed` deletes and recreates only the current shop's data
Public statements (P2)	A token maps to one customer in one shop and exposes read-only fields only
Logs	Include `shopId` in every log line; never log passwords, hashes or tokens

3.14.4 Frontend auth rules

- Next.js `middleware.ts` redirects any `(app)` route to `/login` when the session cookie is absent (a cheap check); the layout then calls `/api/auth/me`, and a 401 clears client state and redirects.
- Hold `user` and `shop` in one React context. Any API 401 anywhere triggers the same redirect, preserving the page the user wanted.
- Never store tokens in `localStorage`; the httpOnly cookie is the only credential.
- Login and register share one `AuthForm` component, with a link to switch.

3.14.5 Isolation test script (run before the demo)

1. Register shops A and B in two browsers (or one normal and one private window).
2. In A, say "Ramesh ne 500 udhaar liya". B's customers list stays empty.
3. In B, call `GET /api/customers/<A's Ramesh id>`: expect 404.
4. In B, say "Ramesh ka kitna baaki hai?": expect the "customer nahi mila" path, not A's balance.
5. Keep B's SSE open while A writes: B receives nothing.
6. In B, call `DELETE /api/commands/<A's groupId>`: expect 404, and A's data unchanged.
7. In B, run "Demo data bharein": A's data is untouched.

3.15 Command history and deletion semantics

The Commands screen lets a shopkeeper review everything they said and delete any command. Deleting a command means **undoing everything it did**, not just hiding a row. One service function implements this, and both the screen's delete button and the voice phrase "last wala hata do" call it.

3.15.1 What is stored

Every voice or typed command writes one ActionLog (see 3.4), including read-only ones such as balance queries. Writes created by that command all carry the same `groupId`. The list endpoint returns, per command, an `effects` summary (`{ entries, reminders, stock, newCustomers }` counts) so the confirm dialog can say exactly what will go away without a second request.

3.15.2 Delete algorithm (`DELETE /api/commands/:groupId`)

1. Find the ActionLog with `{ shopId, groupId, deletedAt: null }`. If none: `404`. Deleting twice returns `200` with all counts zero, so the operation is idempotent.
2. Soft delete Entries with that `groupId` (set `deletedAt`), then recompute balances for every affected customer from the remaining entries.
3. Reminders: `pending` ones become `cancelled`. `sent` ones stay `sent` (an email can't be recalled) and are reported in the response so the UI and voice reply can say so.
4. Stock items created by that command are removed; items whose status the command changed are restored to the previous status.
5. Customers created by the command that now have no remaining entries are archived (`archivedAt`), not hard deleted.
6. Set `deletedAt` on the ActionLog and emit `ledger.updated` on the shop's SSE stream.
7. Return `{ entriesRemoved, remindersCancelled, remindersAlreadySent, stockReverted, customersArchived }`.

3.15.3 Rules

- **Independent of order**: entries are independent, append-only amounts, so deleting an older command is safe; balances are recomputed, never adjusted by subtraction.
- **No hard deletes**: rows stay in the database for audit and debugging. The default list hides deleted commands.
- **Read-only commands** (queries, summaries) have no effects; deleting one just hides it.

- **Spoken confirmation** after a voice undo mirrors the response: "Pichla command hata diya. 2 entries hat gayi. Ek reminder pehle hi ja chuka hai."
- **Confirm sheet copy** (UI): "Is command ko hata dein? Isse jude 2 entries aur 1 reminder bhi hat jayenge." Buttons: "Haan, hatao" and "Rehne do".
- **Authorization**: the command must belong to the caller's shop, else `404`.
- **P2**: multi-select delete and a "Restore" option within 24 hours.

4. Design Rules

4.1 Principles

1. **One hero action**: the mic button is the product. Everything else supports it.
2. **Show the work**: every voice command yields visible action cards so judges *see* the real action.
3. **Glanceable**: big numbers, high contrast, minimal text. A shopkeeper in bright sunlight must read it.
4. **Warm, not corporate**: it should feel like a trustworthy notebook, not a bank.

4.2 Visual language

- **Theme**: light, paper-warm. Background `#FBF7F0`, surface `#FFFFFF`, ink `#1F1B16`.
- **Brand**: saffron `#E8731A` (primary, mic button), deep green `#1F7A4D` (payments in / positive), brick red `#C23B22` (udhaar out / outstanding), muted brown `#8A7F72` (secondary text).
- **Dark mode**: optional, skip unless time remains.
- **Typography**: Headings *Poppins* 600, body *Inter* 400/500, Hindi/Devanagari fallback *Noto Sans Devanagari*. Numbers use tabular figures. Always format currency as `₹1,250` (Indian grouping via `toLocaleString('en-IN')`).
- **Shape**: radius 16 px cards, 999 px mic button and chips, soft shadow `0 4px 20px rgba(31,27,22,.06)`.
- **Spacing**: 4 px base grid; cards padded 20 px; page gutters 16 px mobile, 32 px desktop.
- **Motion** (Framer Motion, subtle): mic pulses when listening (scale 1 to 1.08, 1.2 s loop), action cards slide up with 60 ms stagger, balances count up on change, success check draws in. No motion over 400 ms.

4.3 Layout (mobile first, responsive to desktop)

There are no tabs. After login, Home is an orbital carousel of big glassmorphic tiles, one visible at a time (mic, money records, customers, command history, plus stock, reminders, daily hisaab and settings). Each tile opens a full-screen glass panel with a back chip. The full layout, tile specs and screen designs are in 4.8, which replaces the earlier tab-based layout.

4.4 Component rules

- **ActionCard**: left colored bar by type (red udhaar, green payment, blue reminder, amber stock), icon, one-line summary, status chip (Done / Sent / Failed). Reminder cards show a small email-sent preview.
- **Customer row**: avatar initial, name, balance in red if positive, green if zero or credit; tap opens drawer with entries.
- **Empty states**: friendly sentence plus an example phrase the user can tap.
- **Error states**: never show raw errors; show "Dobara try karein" with a retry button.
- **Accessibility**: min touch target 48 px, contrast AA, `aria-live="polite"` on transcript and results, visible focus rings, don't rely on color alone (add +/- signs and icons).

4.5 Voice UX rules

- Replies are **one or two short sentences**, under 20 words each.
- Reply language mirrors the user's language (Hinglish stays Hinglish, written in Roman or Devanagari consistently so TTS reads well; use Devanagari for `hi-IN` TTS).
- Always **confirm what was done**, include customer name and amount.
- On ambiguity ask **one** question at a time, then auto-listen.
- Never speak while the user is speaking; stop TTS instantly when the mic is pressed.

4.6 Copy tone

Respectful and simple. Use "aap". Examples: "Ho gaya.", "Ramesh ka 500 udhaar likh diya.", "Kaunsa Suresh? Suresh Gupta ya Suresh Yadav?", "Kuch gadbad ho gayi, dobara bolein."

4.7 Design principles in depth

4.7.1 Principles with concrete rules

1. **Voice first, screen second.** The user should be able to finish a task without looking. So every outcome is spoken, and the screen only confirms. Never require a tap after speaking, except to retry an error.
2. **Show system status at all times.** Five states are always visible and visually distinct: Idle, Listening, Thinking, Done, Needs-answer. If more than 400 ms passes with no visible change, something is wrong with the design.
3. **Trust through transparency.** Show the heard transcript, then the understood actions. If the transcript is wrong, the user sees why the result is wrong and can retry. Never hide what the app did.
4. **Forgiveness over confirmation.** Don't ask "Are you sure?" for normal entries; make undo one phrase or one tap instead. Ask only when the app truly cannot decide (ambiguity or missing amount).
5. **Numbers are the heroes.** Money values are the largest, boldest text in any card. Use tabular figures, Indian digit grouping and a rupee sign everywhere.
6. **Plain and respectful language.** Short words, "aap" form, no jargon like "entry" or "transaction" in Hindi mode (use "hisaab", "likh diya", "jama").
7. **One primary action per screen.** The mic is always the biggest, brightest control. Secondary controls are small, outlined and grey.
8. **Consistent color meaning.** Red always means money owed to the shop, green means money received, amber means attention (stock, pending), blue means a system action (reminder, email). Never reuse these colors decoratively.
9. **Honest demo mode.** Anything simulated is labeled with a small "Demo" chip, for example the WhatsApp preview.

4.7.2 Design tokens

Put these in `globals.css` and expose them in the Tailwind config so the AI editor never invents colors:

```
:root {
  --bg: #FBF7F0;          --surface: #FFFFFF;     --ink: #1F1B16;
  --muted: #8A7F72;       --line: #E9E2D6;
  --brand: #E8731A;       --brand-ink: #FFFFFF;   --brand-soft: #FDEBDA;
  --owed: #C23B22;        --owed-soft: #FBE6E1;   /* udhaar, outstanding */
  --received: #1F7A4D;    --received-soft: #E2F3E9; /* payments in */
  --attention: #B7791F;   --attention-soft: #FBF0D9; /* stock, pending */
  --system: #2B6CB0;      --system-soft: #E3EEF9;   /* reminders, email */
  --radius-card: 16px;    --radius-pill: 999px;
  --shadow-card: 0 4px 20px rgba(31,27,22,.06);
  --shadow-mic: 0 10px 30px rgba(232,115,26,.35);
}
```

Token group	Rule
Type scale	Display 32/40 (stat numbers), H1 24/32, H2 20/28, Body 16/24, Small 14/20, Caption 12/16; Poppins 600 for headings and numbers, Inter for body
Spacing	4 px grid: 4, 8, 12, 16, 20, 24, 32, 48. Card padding 20, gap between cards 12
Radius	Cards 16, inputs 12, chips and mic 999
Elevation	Only two levels: card shadow and mic shadow. No borders plus shadows together
Icons	lucide-react, 20 px in cards, 24 px in nav, stroke 1.75, never filled
Breakpoints	0-639 mobile (single column), 640-1023 tablet, 1024+ desktop two columns

4.7.3 Interaction states

Every interactive element needs all of these states defined, not just default:

Element	Default	Active or listening	Loading	Disabled or error
Mic button	Saffron, soft shadow, label "Dabake bolo"	Scale 1.08 pulse, ripple ring, label "Sun raha hoon..."	Spinner ring, label "Soch raha hoon..."	Grey with slash icon and "Mic allow karein"
Transcript bubble	Empty hint with one example phrase	Grey interim text, caret blinking	Final text locked in ink	Red outline and "Sunai nahi diya"
Action card	Colored left bar, icon, summary	Slide-up with 60 ms stagger	Skeleton shimmer	Red chip "Failed" with Retry
Customer row	White, balance right-aligned	Pressed state brand-soft	Skeleton line	Muted with "Load nahi hua"
Stat card	Number and label	Count-up when value changes	Grey bar placeholder	Dash with tooltip

4.7.4 Motion rules

- Motion explains cause and effect: a card slides up because it was just created, a number counts up because it changed. No decorative loops except the listening pulse.
- Durations: micro 120 ms, standard 220 ms, emphasis 360 ms. Easing: `easeOut` in, `easeIn` out.
- Respect `prefers-reduced-motion`: replace slides and pulses with simple fades.
- Never animate layout properties of large lists; animate `transform` and `opacity` only.

4.7.5 Accessibility rules

- Contrast AA for all text; large numbers also AA on tinted backgrounds. Check `--muted` on `--bg` (it must be at least 4.5:1 for body text, otherwise darken it).
- Touch targets at least 48 by 48 px; mic button 72 px.
- Never rely on color alone: red and green values also carry a sign (+ or -) and an icon.
- `aria-live="polite"` on the transcript and on the results list; the mic button has `aria-pressed` and a clear label.
- Full keyboard path: Space or Enter on the mic, Tab through cards, Escape closes drawers.

- Offer a big-text option (125 percent) in the language toggle menu, because many shop owners are over 40.

4.7.6 Microcopy matrix

Moment	Hinglish	English
Idle hint	"Dabake bolo"	"Hold and speak"
Example chip	"Ramesh ne 500 udhaar liya"	"Ramesh took 500 on credit"
Thinking	"Soch raha hoon..."	"Working on it..."
Success	"Ho gaya."	"Done."
Clarify	"Kaunsa Suresh?"	"Which Suresh?"
No speech heard	"Sunai nahi diya, dobara bolein"	"Didn't catch that, please repeat"
Parse failed	"Samajh nahi aaya, dobara bolein"	"I didn't understand, please try again"
Network or model error	"Thodi gadbad hai, dobara try karein"	"Something went wrong, try again"
Empty ledger	"Abhi koi udhaar nahi. Bolkar shuru karein"	"No credit yet. Speak to start"
Undo	"Pichla entry hata diya"	"Last entry removed"

4.7.7 Design do and don't

- Do keep the demo screen uncluttered: stats, console, one list. Judges read it from 3 meters away.
- Do pre-load seed data so no screen is ever empty during the demo.
- Keep settings to one small glass sheet (shop name, language, big text, logout). No theme editors or extra profile screens; they cost time and score nothing.
- Don't show raw JSON, IDs or timestamps in the main UI; keep them in the `/debug` page.
- Don't use more than two font weights per card.

4.8 Glassmorphic launcher UI

This section replaces the tab layout. The whole app is one visual system: a soft gradient backdrop with frosted-glass surfaces on top. Navigation is an orbital carousel of big glass tiles, one visible at a time, moved by swipe or arrows, never a tab bar.

4.8.1 Backdrop and glass tokens

Glass only looks like glass when there is color behind it, so the backdrop is a warm gradient mesh. Add these to `globals.css` next to the tokens in 4.7.2:

```
:root {
  --backdrop:
    radial-gradient(60% 50% at 15% 10%, #FFD9B0 0%, transparent 60%),
    radial-gradient(50% 45% at 90% 20%, #BFE6D0 0%, transparent 60%),
    radial-gradient(55% 50% at 70% 95%, #F6C6B8 0%, transparent 60%),
    #FBF7F0;
  --glass-bg: rgba(255,255,255,0.55);
  --glass-bg-strong: rgba(255,255,255,0.80);
  --glass-border: rgba(255,255,255,0.70);
  --glass-shadow: 0 8px 32px rgba(31,27,22,0.10), inset 0 1px 0
rgba(255,255,255,0.8);
  --glass-blur: 18px;
  --radius-tile: 28px;
}
body { background: var(--backdrop); background-attachment: fixed; color:
var(--ink); }
.glass {
  background: var(--glass-bg);
  backdrop-filter: blur(var(--glass-blur)) saturate(160%);
  -webkit-backdrop-filter: blur(var(--glass-blur)) saturate(160%);
  border: 1px solid var(--glass-border);
  box-shadow: var(--glass-shadow);
  border-radius: var(--radius-tile);
}
.glass-strong { background: var(--glass-bg-strong); }
@supports not (backdrop-filter: blur(1px)) { .glass { background: var(--
glass-bg-strong); } }
@media (prefers-reduced-transparency: reduce) { .glass { background: #fff;
backdrop-filter: none; } }
```

4.8.2 Home: orbital tile carousel

Home shows **exactly one huge glass tile at a time**. The tiles sit on an invisible elliptical orbit: moving to the next tile swings the current one around the back of the orbit while the next swings in at the front. The user moves with a swipe on phones, with arrow buttons, or with keyboard arrows, and the carousel loops forever. Order: Mic, Money Records, Customers, Commands, Today's Summary, Stock, Reminders, then back to Mic.

```
        . - ~ ~ - .               faint orbit ring (decorative)
     /              \
( < )   +-----------+   ( > )     arrow buttons: 64 px glass circles
        |           |
        |   (MIC)   |             ONE tile, about 340 x 420 px on phones
        |  Dabake   |
        |   bolo    |
        +-----------+
         3 / 7  Customers          position and name
        o  o (O) o  o  o  o        orbit dots, active one lit
 Latest: "Ramesh ne 500 ..." -> 4  activity strip (glass, one line)
```

Interaction rules

Input	Behavior
Swipe left or right	The tile follows the finger; on release it snaps to the nearest tile, using position and velocity (a flick above 500 px/s always advances one step). Never skip more than one tile per gesture. Use `touch-action: pan-y` so vertical page scroll still works
Arrow buttons	Glass circles, 64 px (72 px on desktop), vertically centered on the tile near each screen edge. One step per press, loops at both ends, 450 ms cooldown so a double tap doesn't skip a tile. Labels: "Pichla" and "Agla"
Keyboard	Left and Right arrows move; Home jumps to Mic; Enter or Space opens the active tile
Trackpad or wheel (horizontal)	One step per gesture, with the same 450 ms lock
Tap or click the tile	Opens its panel. On the Mic tile, hold to talk; a quick tap only shows the hint "Dabake kar bolo"

Input	Behavior
Orbit dots	Tap a dot to jump to that tile; the orbit rotates the shortest way around
Launch and return	The app always starts on Mic. After closing a panel, the carousel returns to the tile that was open
Mic shortcut	When the active tile is not Mic, a floating mini mic (56 px glass orb, bottom-right) is shown, so speaking is always one press away

Size and content of the active tile

- Phones: card width `min(86vw, 340px)`, height `min(58vh, 430px)`, radius 36 px. Desktop: 420 by 480 px, arrows 72 px outside the card with a 32 px gap.
- Contents, top to bottom: tinted glass orb with the icon (140 px orb, 88 px icon; Mic orb 200 px), label (Poppins 600, 28/36), live badge (Display 48/56), one hint line (14 px, muted), such as "Tap karke kholein".
- Below the card: "3 / 7 Customers" in 14 px, the orbit dots, and the one-line latest-activity strip.

Orbit motion

Keep one fractional index `p` (the carousel position) and compute every tile from it. With `N` tiles and `d` the signed shortest distance from tile `i` to `p` on the loop:

Property	Formula
`d`	`((i - p + N/2) mod N) - N/2`
`theta`	`d * 2*PI / N`
`x`	`sin(theta) * Rx`, with `Rx = min(0.62 * viewportWidth, 380)` px
`y`	`(1 - cos(theta)) * 48` px (the tile dips as it travels the far side)
`scale`	`0.5 + 0.5 * max(cos(theta), 0)`
`rotateY`	`-d * 28` degrees

Property	Formula
`opacity`	`clamp(1 - 1.3 * abs(d), 0, 1)` so neighbors are invisible at rest and cross-fade during the move
`zIndex`	`round(cos(theta) * 10)`

```
const N = tiles.length;
const p = useMotionValue(0);                          // fractional carousel
index
const go = (to: number) =>
  animate(p, to, { type: 'spring', stiffness: 220, damping: 26 });   // about
420 ms

function OrbitTile({ i, children }: { i: number; children: React.ReactNode })
{
  const d       = useTransform(p, (v) => ((((i - v + N / 2) % N) + N) % N) -
N / 2);
  const x       = useTransform(d, (v) => Math.sin((v * 2 * Math.PI) / N) *
Rx);
  const y       = useTransform(d, (v) => (1 - Math.cos((v * 2 * Math.PI) /
N)) * 48);
  const scale   = useTransform(d, (v) => 0.5 + 0.5 * Math.max(Math.cos((v * 2
* Math.PI) / N), 0));
  const rotateY = useTransform(d, (v) => -v * 28);
  const opacity = useTransform(d, (v) => Math.max(0, 1 - 1.3 * Math.abs(v)));
  return (
    <motion.div style={{ x, y, scale, rotateY, opacity, position: 'absolute',
perspective: 900 }}>
      {children}
    </motion.div>
  );
}
// Drag: map 0.8 * cardWidth of finger travel to one index unit,
// then on release call go(Math.round(p.get() + velocityBoost)).
```

- **Orbit dots**: draw a small ellipse (about 160 by 36 px) under the card with `N` dots at `(sin(theta) * 80, cos(theta) * 18)`; the dot nearest the front is larger and saffron. The dots rotate as `p` changes, which echoes the orbit above.
- **Decorative ring**: a faint dashed ellipse behind the tile (white at about 18 percent opacity). It doesn't move; the tile does.

- **Reduced motion**: skip the orbit; crossfade between tiles in 200 ms with no spring.
- **Performance**: only a tile with opacity above 0.02 gets the `.glass` blur (set `visibility: hidden` and drop the class otherwise), so at rest exactly one blurred tile is on screen and at most two mid-move.
- **Accessibility**: the container has `role="region"` and `aria-roledescription="carousel"`; each tile is `role="group"` with `aria-label="3 of 7: Customers"`; only the active tile is focusable (the others `aria-hidden` and `tabindex=-1`); an `aria-live="polite"` region announces the new tile. Arrow buttons give swipe-free access.
- **Haptics**: `navigator.vibrate(8)` on each snap where supported.

4.8.3 Tile specification

Tile	Lucide icon	Orb color	Badge	Notes
Mic ("Bolo")	`Mic`	Saffron `--brand`	none; pulse while listening	Default tile at launch; hold to talk; orb 200 px
Money Records	`Wallet`	Green `--received`	Today's net with sign	Opens the money feed
Customers	`Users`	Red `--owed`	Total outstanding	Opens the customer khata
Commands	`AudioLines`	Blue `--system`	Today's command count	Opens history with delete
Today's Summary	`Sun`	Amber `--attention`	Today's net	P1
Stock	`Package`	Amber `--attention`	Low-stock count	P1
Reminders	`BellRing`	Blue `--system`	Pending count	P1
Settings	`Settings`	Neutral	none	Small glass icon in the top bar

All tiles share these rules:

- Surface: `.glass`, radius 28 px, padding 20 px, size as in 4.8.2 (about 340 by 420 px on phones).
- Icon: 88 px, drawn inside a 140 px frosted orb (Mic: 200 px) tinted with the tile's color at about 25 percent opacity, with a soft colored glow behind it.
- Label: Poppins 600, 28/36, in the user's language. Badge: Display 48/56, tabular figures, colored by meaning (green for gains, red for owed).
- States: hover lifts 2 px and brightens the border; press scales to 0.97 with a 120 ms ease; focus shows a 3 px saffron ring; a loading badge shows a shimmer bar.
- Badges update live from SSE; a changed badge counts up and flashes once.

4.8.4 Mic tile behavior

- Press and hold anywhere on the mic tile to record; the orb scales to 1.08, rings ripple outward, and the label becomes "Sun raha hoon...". Vibrate for 10 ms on press and on success where `navigator.vibrate` exists.
- On release the label becomes "Soch raha hoon..." and the orb shows a slow spinning ring.
- Results appear in a **result sheet**: a glass bottom sheet about 60 percent high with the transcript, the action cards stacked, and the spoken reply as text. Swipe down or tap outside to dismiss; it also auto-minimizes after 8 seconds of idle.
- After a successful write command, show an **Undo toast** for 5 seconds ("Hatao"). It calls the same delete service as the Commands screen (3.15).

4.8.5 Screen panel pattern

Every tile opens the same container so the app feels like one object:

- `GlassPanel` fills the screen, with the backdrop visible in a 12 px margin on desktop and edge to edge on mobile.
- Sticky glass header: back chip ("< Ghar") on the left, screen title in the middle, optional action on the right (filter, search).
- Entry animation: the tile expands into the panel using a Framer Motion shared `layoutId` over 320 ms; back reverses it. With reduced motion, use a 150 ms fade.
- Content scrolls inside the panel. List rows are solid white at 0.9 opacity (not blurred) so long lists scroll smoothly and text stays readable.
- A floating **mini mic** (56 px glass orb, bottom-right) is present in every panel, so the user can speak from anywhere. The result appears in the same result sheet, and the panel's data refreshes through SSE.

4.8.6 Screen details

- **Money Records**: header glass card with three numbers (in, out, net) for the selected range; filter chips below (Sab, Udhaar, Jama, Aaj, Hafta, Mahina); the feed grouped by day with signed colored amounts.
- **Customers**: sticky search glass bar (with an inline mic); cards sorted by amount owed; tapping opens a drawer from the right (bottom sheet on mobile) with that customer's ledger and "Yaad dilao".
- **Commands**: rows show the quoted transcript, action chips, time and status. Delete is a trash icon on each row on desktop, and a swipe-left reveal on mobile, both opening the glass confirm sheet from 3.15.3. The row slides out on success and the balances elsewhere update live.
- **Confirm sheet**: small centered glass card over a dimmed backdrop, one sentence, two buttons: a red filled "Haan, hatao" and a plain "Rehne do". Focus starts on "Rehne do".
- **Login and Register**: one centered glass card (max width 420 px) on the gradient backdrop; logo orb on top, title, 56 px inputs, a primary saffron button, and a text link to switch between the two. A language toggle sits in the card's top-right corner. No tabs, no side art.

4.8.7 Legibility, performance and accessibility for glass

- Text never sits on bare `--glass-bg` over busy color: use `.glass-strong` for any card carrying small text or money values, and keep ink `#1F1B16` on it (contrast at least 4.5:1; verify with the backdrop at its brightest area).
- At most two stacked glass layers on screen (for example a panel plus a sheet). No glass inside glass inside glass.
- Keep about 12 or fewer `backdrop-filter` elements visible at once. On low-end Android, drop `--glass-blur` to 12 px through a `data-perf="low"` attribute set when `navigator.hardwareConcurrency` is 4 or below.
- Respect `prefers-reduced-transparency` and `prefers-reduced-motion` (see the CSS above and 4.7.4).
- Never place red or green text directly on colored glow; put it on a white card.
- Hit targets stay at least 48 px, and all tiles are real `<button>` or `<a>` elements with labels so screen readers and keyboards work.

4.8.8 Extra touches I added

These cost little and make the demo feel finished:

- **Floating mini mic** on every panel, so voice is always one press away.
- **Undo toast** after each command, matching the delete semantics in 3.15.
- **Latest activity strip** on Home that shows the last command and its action chips; tapping it opens that command in Commands.
- **Rotating example chip** under the mic ("Bolo: Ramesh ne 500 udhaar liya", "Bolo: aaj ka hisaab"), tappable to run it as a typed command.
- **Live badges** on tiles through SSE, with a count-up when they change.
- **Skeleton shimmer** on tiles and rows while data loads, never blank glass.
- **Haptic feedback** on mic press and on success.
- **Greeting by time** ("Suprabhat", "Namaste", "Shubh sandhya") using the shop owner's name.

5. Six-Hour Execution Plan (solo)

Time	Goal	Done when
0:00-0:30	Kickoff. Create Atlas cluster, API keys, repos	Env vars ready
0:30-1:00	Idea freeze (this doc). Scaffold monorepo and shop-scoped auth with AI editor	Both servers run
1:00-2:00	**Core loop**: mic -> transcript -> `/voice/command` -> one action (udhaar) -> TTS	Say a sentence, hear reply, see DB row
2:00-3:00	Multi-intent, payments, reminders (email), resolver	Section 1 sentence works
3:00-3:30	Mentor check-in. Add queries, summary, clarification	U7, U9, U11
3:30-4:30	Glass Home tiles, Records, Customers, Commands with delete, SSE, undo, seed data	UI polished enough
4:30-5:00	Bug bash with test script, rehearse demo, record backup video	3 clean runs
5:00-6:00	Demos	

Cut order if behind: statement link, then stock, then SSE (use polling), then undo, then clarification. Never cut the core loop or the email action.

6. The 3-Minute Demo Script

- **0:00-0:20 Hook**: "Raju runs a kirana store. His hands are full of customers, his udhaar is in a notebook." Show messy bahi photo (optional) then BolBahi dashboard with seeded data.
- **0:20-1:10 Magic moment**: hold mic, say the multi-intent sentence. Judges see four cards appear, balance numbers count up, the app speaks the summary.
- **1:10-1:40 Real action**: open the inbox on a second screen, show the reminder email that just arrived.
- **1:40-2:15 Intelligence**: "Ramesh ka kitna baaki hai?", then an ambiguous name that triggers the spoken clarification.
- **2:15-2:40 Recovery**: "Last wala hata do" undoes it. Then "Aaj ka hisaab".
- **2:40-3:00 Honesty and impact**: "Real: STT, Claude parsing, MongoDB, email. Simulated: WhatsApp. Next: offline mode, regional languages, UPI payment links."

Have a typed fallback and a pre-recorded 60 s video in case the venue Wi-Fi or mic fails.

7. Risks and Mitigations

Risk	Mitigation
Web Speech mishears Hinglish	Prompt tells the LLM transcripts may be noisy; fuzzy name resolver; demo with clear short sentences; typed fallback
Noisy hall mic	Use headset mic for the demo
LLM latency	Flash-class model, small context, show "Soch raha hoon..." state
API outage / Wi-Fi	Backup video, local run, phone hotspot
App hears its own TTS	Stop recognition during speech
Scope creep	Follow the cut order above

8. Master Brief for the AI Code Editor (paste this first)

```
You are building "BolBahi", a voice-first khata (credit ledger) web app for
Indian
kirana shopkeepers, for a 6-hour solo hackathon. Priority: a WORKING demo
over
perfection. Build in the order given and keep every step runnable.

STACK
- apps/web: Next.js (App Router, TypeScript), Tailwind, shadcn/ui, Framer
Motion, lucide-react
- apps/api: Express + TypeScript on Node 20, Mongoose (MongoDB Atlas), zod,
Fuse.js,
  node-cron, express-rate-limit, Resend (or nodemailer), @google/genai
- LLM: Google Gemini (Flash model, free tier) using forced function calling.
The model name
  comes from the GEMINI_MODEL env var, never hardcoded. ALL LLM access goes
through an
  LLMProvider interface: parse(transcript, context) -> ParseResult. Default
provider is
  gemini; a Claude provider can be added later and selected with
LLM_PROVIDER=gemini|claude.
- STT: browser Web Speech API (hi-IN / en-IN). TTS: browser SpeechSynthesis.
- Realtime: SSE at GET /api/events (one stream per shop).
- Auth: register/login with bcryptjs + JWT in an httpOnly cookie. EVERY model
has shopId and EVERY
  query filters by req.shopId set by requireAuth. Never trust a shopId sent
by the client.

CORE FLOW
Mic (hold to talk) -> live transcript -> POST /api/voice/command -> Gemini
function call
(record_actions) returns ordered actions -> zod validate -> executor writes
to Mongo,
sends reminder email -> response {actions, results, reply,
needsClarification}
-> UI shows action cards and speaks `reply` -> SSE updates dashboard.

FOLLOW EXACTLY the data models, action schema, API contract, executor rules,
system design
rules (TRD 3.13) and folder structure in this doc. Follow the Design Rules
(saffron #E8731A
primary, paper background #FBF7F0, green #1F7A4D, red #C23B22, Poppins +
```

```
Inter + Noto Sans
Devanagari, 16px radius, mobile-first, 72px mic button, aria-live regions).

SYSTEM RULES (non-negotiable)
- Layering: routes (thin, validate, respond) -> services (business logic) ->
models. No business
  logic in routes. The LLM only PARSES; it never writes to the DB or sends
email itself.
- Treat LLM output as untrusted input: validate with zod, clamp amounts to
1..1,000,000,
  reject negatives, cap actions per command at 8, ignore unknown action
types.
- Gemini calls: temperature 0.1, 8s timeout, retry up to 2 times with backoff
(400ms, 1200ms)
  on 429/503, then return a friendly spoken error. Never crash the request.
- One uniform error envelope: { ok:false, error:{ code, message,
spokenMessage } }.
- Idempotency: the client sends clientCommandId; a repeated id returns the
stored result
  instead of writing twice.
- Process commands per sessionId one at a time (simple in-memory promise
queue) to avoid races.
- Recompute customer balance from entries after every write. Soft delete
only.
- Email failure must not fail the ledger write; report it in results[].
- Free-tier privacy: send only the transcript and up to 30 customer first
names to Gemini.
  Never send phone numbers or emails. Use fake data in demos.
- Structured logs (one line JSON) with groupId, latencyMs, actionTypes,
provider.
- Never expose API keys to the client. Cap transcript length at 500 chars.

BUILD ORDER (stop and confirm each runs before the next)
1. Scaffold monorepo, env handling (zod-validated env), Mongo connection,
health route.
   Then auth: User + Shop models, /api/auth/register|login|logout|me,
requireAuth middleware,
   login and register pages, route guard on (app) pages. Verify two shops
never see each other's data.
2. Models + POST /api/demo/seed for the logged-in shop (10 Indian customers
incl. two named Suresh, balances, 2 stock items).
3. LLMProvider interface + Gemini implementation + voice.ts route + executor
for add_udhaar
```

```
   only. Test with curl.
4. Frontend: MicButton with Web Speech, transcript, typed fallback, TTS, call
the API, show a
   basic result.
5. Add add_payment, set_reminder (immediate email + node-cron scheduled),
add_low_stock.
6. Add resolver (exact, alias, fuzzy), clarification loop with 60s pending
state per sessionId.
7. Add query_balance, daily_summary, undo_last (soft delete by groupId).
8. Orbital tile carousel (glass tiles with live badges, swipe + arrows),
Records, Customers, Commands History with delete,
   Stock, Reminders, Aaj ka Hisaab, Settings sheet, SSE live refresh.
9. Polish: animations, empty states, error states, Hindi/English toggle, "Run
demo sentence"
   button, loading and listening states.

UI LAYOUT (see Design 4.8)
- No tabs. After login, Home is an orbital carousel showing ONE huge glass
tile at a time (swipe on phones, arrow
  buttons, keyboard arrows; loops; tiles travel along an elliptical orbit,
see 4.8.2): Mic (default), Khata Records,
  Customers, Commands History, Stock, Reminders, Aaj ka Hisaab, Settings.
Each tile opens a
  full-screen glass panel with a back chip. Tiles show live badges
(outstanding total, count).
- Commands History lists past voice commands. Deleting one asks for a single
confirm, then undoes
  its effects (soft delete by groupId) and removes it from the list.
- Login and Register pages share one glass card. Unauthenticated users are
redirected to /login.

UX RULES
- Replies: max 2 short sentences, mirror the user's language, always include
name and amount.
- Pause speech recognition while TTS is speaking; stop TTS when the mic is
pressed.
- Every voice command yields visible action cards (type color bar, icon,
summary, status chip).
- Every async state has a visible UI: listening, thinking, done, failed,
needs-clarification.

DEFINITION OF DONE
- The 7-step test script in TRD 3.12 passes.
```

```
- The demo sentence yields 4 action cards, a real email, an updated dashboard
and a spoken reply.
- No unhandled promise rejections; no raw errors in the UI.
- After each step, show me the commands to run and a one-line manual test.
```

Handy follow-up prompts for the editor

- "Write the Gemini system prompt and function declaration in llm.ts exactly per TRD 3.5 and 3.6, with 10 few-shot Hinglish examples including numbers like 'dedh sau' and 'paanch sau'."
- "Create the SpeechRecognition wrapper in speech.ts with hold-to-talk, interim results, silence auto-stop, and clean error callbacks."
- "Build the ActionCard component with type-based color bar, icon, status chip, and slide-up stagger animation."
- "Write a Jest or script-based test that runs the 7-step test script in TRD 3.12 against the API."

9. Quick Checklist Before You Present

- [ ] Seed data reset, second screen shows the inbox
- [ ] Chrome, HTTPS or localhost, mic permission pre-granted
- [ ] Headset mic tested in the venue
- [ ] Backup video recorded, typed fallback ready
- [ ] README states what is real and what is simulated
- [ ] Rehearsed to land at 2:50

Good luck. Get the core loop working in the first two hours and everything else is bonus.